Comprehensive Educational and Paper Trading Platform

CSCI 455/555: Generative AI for Software Development – Final Project

Aidan Basloe & Jeffery Lin
William & Mary

May 6, 2026

Abstract

The proposed project is a comprehensive educational and (paper) trading platform designed to demystify the stock market through real-time AI integration. Unlike traditional brokerages that focus on transaction volume and practical usage, our app positions itself as a testing ground that provides market literacy and strategic planning for retail traders. By utilizing an interactive UI, users can explore real-time data for major indices and a multitude of other diversified securities. We provide a tutorials section dedicated to explaining the basics of investment markets, and integrate AI explanations on-demand for each important feature. Users will also have an interface for implementing and testing LLM-integrated strategies and models. The goal is to provide a low-friction entry point for beginners while offering sophisticated modeling tools for experienced investors.

1 Introduction

Financial instrumentation is an essential part of our national economy. One survey found that more than 60% of adults in the USA have money invested into the stock market, either directly on individual stocks or indirectly through retirement plans like the 401k and Roth IRA. Of those who do day trade, a high percentage are not "beating the market," which most define as exceeding the average tax-advantaged 10% annual returns of the S&P 500. From these statistics, we can infer that a large population of individuals aren't especially interested or educated in the particularities and inner workings of investing or financial markets in general.

Furthermore, there is a large disconnect between how institutional "players" function in the trading world and what ordinary people know about the stock market. This divide coalesces in cultural flashpoints like the rise of meme stocks during the early 2020s, movies like The Big Short and The Wolf of Wall Street, or public perceptions of figures like Warren Buffet and Nancy Pelosi. Although a lot of ordinary people have newly invested in the stock market, facilitated by commission-free trading apps like Robinhood and coordinated via

social media forums like Reddit's r/wallstreetbets, retail investors and the general public's comprehension of the stock market are still somewhat sensationalized.

In order to fix the knowledge gap, we want to democratize not just the access to trading, but the literacy of the market's mechanical risks. By demystifying concepts like market-making, dark pools, etc. we can empower individuals to move beyond "vibe-based" investing. We also want to enable users to learn, analyze, and apply trading methods on the stock market in real time, leveling the playing ground so that all traders can strategize and practice. Understanding these nuances ensures that the next generation of investors isn't just participating in the market, but is doing so with a clear-eyed understanding of the rules that govern the playground.

2 Market Analysis

2.1 Target Segment

Our web app targets newer traders, including people who have just started to learn about the stock market, people new to the workforce, and younger people who want to learn about investing. For these people, our app would be advantageous as it is a vertically integrated training and testing grounds for those interested in investing and the stock market.

2.2 Problem Justification

Existing apps have barriers to entry like subscription or membership fees. Moreover, we want our app to be an all-in-one solution for those learning about the stock market and applying those lessons in paper or actual trading. By combining features that are both functional and educational, we create a novel web app which is simple and customizable for newer financial traders or people interested in learning about the stock market. Finally, we integrate AI for both the educational and the functional side of our app to provide our users the edge of newer AI models.

2.3 next

In general, we believe our web app would facilitate more efficient markets and more educated and technically proficient investors. Since there are a large number of companies in the fintech space we know there is a proven business model. For example, KPMG International states that $116 billion worth of funding and deals were made around fintech startups in 2025, and that is not counting the colossal amount of revenue that existing fintech companies generate. We think our concept could potentially capture a share of the fintech market space since we provide some novel features.

In particular, we want to implement a trading-testing-analysis system which would help to automate the quantitative modelling/testing process. In specific, we want to have a Python-based "Strategy Builder". This tool will allow users to translate their natural language trading ideas into functional Python code via a GUI, either to plug into our paper-trading system or an external API/broker. We also want to have a clean and effective tutorial section

detailing the fundamental/necessary knowledge needed for new investors and those who want to learn more about the stock market. We think there is market value in structuring our tutorial section like an infinite-scrolling article with a table of contents, where information can be augmented by on-demand LLM explanations featuring RAG or other helpful architectures.

With regard to the current competitors in our space, we resolve some issues and possess comparative advantages. Several stock market learning or paper-trading related platforms already exist, like Investopedia for example. However, websites like Investopedia often mix real educational content and advertisements/news together, thus decreasing how effectively they can teach unknowledgeable investors. Furthermore, their paper-trading game suboptimally reflects the real market, failing to account for order liquidity and how real traders trade.

Brokerages like Schwab and Robinhood also regularly embed and publish pages meant for investors to learn things, like what an ETF, stock, option, or future is for example. These pages are often hidden away on their website, which creates a barrier to entry and a cognitive separation between learning the stock market and playing the stock market. We believe we can improve on these aspects and turn learning the stock market into something fun.

There are some platforms that already exist which have features similar to our strategy-builder market analyzer, such as Composer or NinjaTrader. But we can have an advantage over these existing apps by having our website not require signing up or paying a fee. While professional traders may afford to use these high cost and complicated tools, we aim for temporal efficiency so that people can quickly build and move from strategy to strategy.

Finally, one primary obstacle for retail investors is the steep learning curve associated with financial terminology. Our solution embeds generative AI directly into the interface. By allowing users to click any on-screen element for an immediate, context-aware explanation, we transform a static dashboard into an active learning environment.

3 System Overview

The GenAI Trading Dashboard is organized as a browser-based single-page application with five main user-facing areas: a home screen, an interactive charting workspace, a paper trading workspace, a strategy builder, and an educational reader. The goal of the system is to keep learning, market observation, strategy testing, and simulated execution in one interface so that a user can move from understanding a concept to testing it against real market data without switching tools.

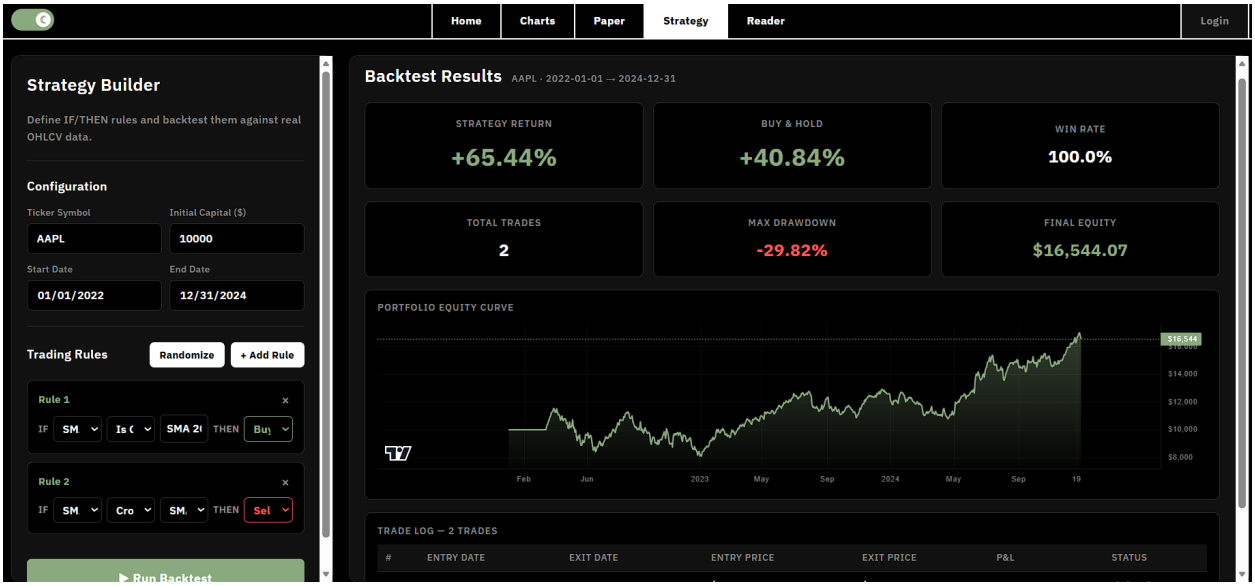
The main navigation is implemented as a tabbed React interface. The current version includes `Home`, `Charts`, `Paper`, `Strategy`, and `Reader` tabs. The tab bar also includes light/dark theme control and a lightweight login form. This layout gives the application a dashboard structure: each tab acts as a focused tool, while shared styling variables and API utilities keep the experience consistent across the platform.

The `Charts` tab is the core market-analysis workspace. It uses `lightweight-charts` to render a TradingView-style candlestick chart with volume bars, crosshair interaction, stock search, dynamic history loading, and periodic background refreshes. The chart initially loads recent OHLCV data for the selected ticker and then requests older six-month chunks as the user scrolls or zooms left. This makes the chart feel continuous without requiring the browser

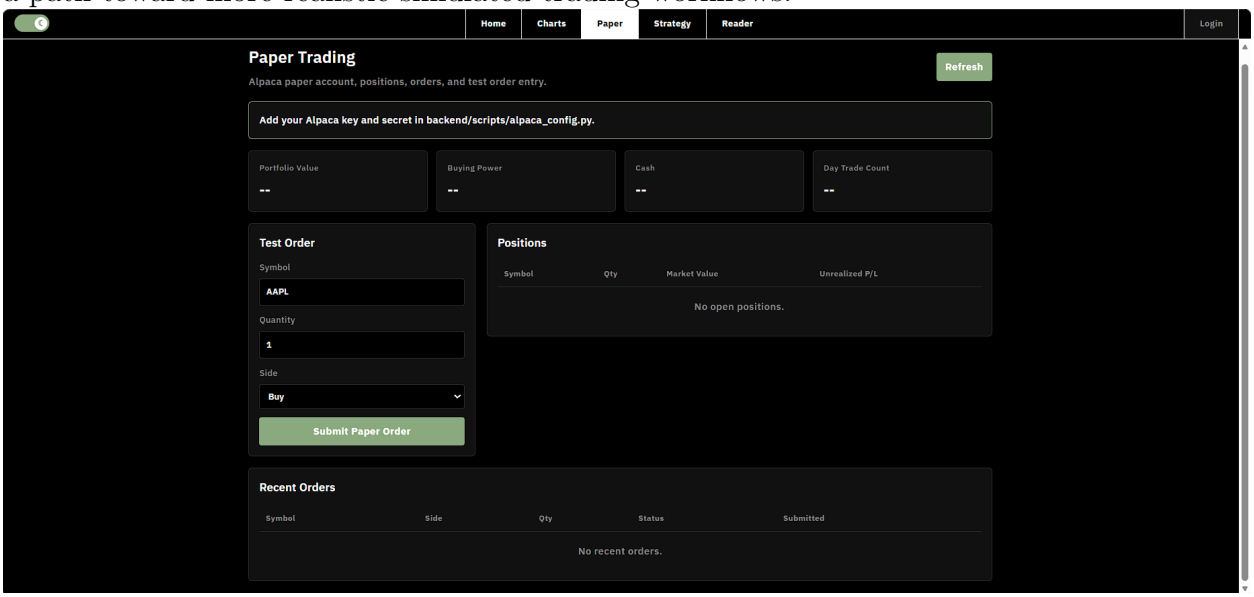
to load all historical data at once. The chart view also includes technical overlays such as moving averages, price range markers, and a volume profile. When Alpaca market data is available, the volume profile can be generated from Alpaca data; otherwise, the frontend falls back to a local estimate based on the visible OHLCV window.

A central feature of the chart workspace is AI-assisted technical analysis. When the user clicks `AI Analyze`, the frontend captures only the currently visible chart window and sends a compact OHLCV payload to the backend. The backend computes summary metrics such as start price, end price, percent change, period high and low, volatility, and average volume, then passes the data to an OpenRouter-hosted LLM. The returned analysis is displayed in a resizable side panel next to the chart. This design keeps the LLM grounded in the same data the user is seeing, rather than asking it to analyze an ambiguous ticker symbol in isolation.

The `Strategy` tab provides a rule-based strategy builder and backtesting interface. Users construct IF/THEN-style rules using supported indicators such as SMA 20, SMA 50, SMA 200, EMA 20, RSI 14, MACD, and Price. Each rule combines an indicator, a condition, a comparison value, and an action such as Buy, Sell, Close Position, or Do Nothing. When the user runs a backtest, the frontend sends the selected ticker, date range, starting capital, and rules to the FastAPI backend. The backend fetches historical OHLCV data, evaluates the rules in a pure-Python backtesting engine, and returns performance metrics, an equity curve, and a trade log. The frontend then renders stat cards, a real equity-curve chart, and a scrollable table of simulated trades.



The `Paper` tab connects the educational and analytical parts of the platform to simulated order execution. It is designed around Alpaca paper trading and displays account information, open positions, recent orders, and a simple order-entry form. The backend centralizes Alpaca API calls so the frontend does not communicate directly with the brokerage API. This keeps the paper-trading functionality separated from the visualization layer and creates a path toward more realistic simulated trading workflows.



The `Reader` tab supplies the educational side of the platform. It presents investing and trading concepts in a structured article format with a table of contents and section-level navigation. The chart workspace can dispatch users directly into relevant reader sections, which supports the project's main learning loop: a beginner can encounter an unfamiliar concept in the live chart, jump to the explanation, then return to the chart or strategy builder with more context.

Architecturally, the system follows a clear frontend-backend split. The React/Vite frontend owns the user interface, chart rendering, tab state, theme state, and client-side interac-

tions. A shared `api.js` module wraps all calls to the backend. The FastAPI backend runs on port 3000 and acts as the integration layer for market data, AI analysis, backtesting, authentication, and Alpaca paper trading. Python scripts in `backend/scripts/` handle specialized work: `stockdata.py` fetches market data, `chart_analyzer.py` prepares and submits LLM analysis requests, `backtest.py` executes the strategy simulation, and `alpaca_config.py` handles paper-trading and market-data requests.

Overall, the system is built around a repeated workflow: fetch real or simulated market data, visualize it in an interactive interface, let the user ask for AI explanation or define a trading rule, and then return measurable results through analysis panels, backtest metrics, or paper-trading state. This workflow is what distinguishes the project from a static educational site or a basic stock chart. The dashboard is intended to function as an integrated training environment where market literacy and strategy experimentation reinforce each other.

4 Tech Stack

For our web application, we adopted a modified MERN-inspired architecture, replacing the traditional Node/Express backend with a Python-based FastAPI server. We chose Python for our backend primarily to leverage its powerful ecosystem for quantitative finance and rapid mathematical calculations. Specifically, utilizing a Python backend allowed us to seamlessly integrate the yfinance library for data aggregation and build a custom rule-based backtesting engine from scratch.

Frontend Development: The frontend was built using React and compiled with Vite. React's component-based architecture was essential for managing the application's complex state and delivering a highly responsive user experience, which is critical for real-time financial dashboards. To visualize the data, we integrated two charting libraries: Chart.js for standard data visualizations and Lightweight Charts (by TradingView) for rendering high-performance, interactive candlestick charts with volume overlays. React's widespread adoption also ensured that our AI coding assistants had deep familiarity with the framework, facilitating a rapid development cycle.

Backend & Database: The backend API layer was powered by FastAPI and Uvicorn, which handled all API orchestrations, data routing, and heavy computational tasks—such as processing historical OHLCV data to calculate technical indicators (SMA, EMA, RSI, MACD) for our Strategy Builder. For user authentication and persistence, we integrated a local MongoDB database to handle account creation and login state securely.

AI Integration: To power the application's interactive educational features and technical analysis capabilities, we integrated a Large Language Model (LLM) directly into the dashboard. We utilized the OpenRouter REST API to connect with the Tencent Hy3 model. By passing visible chart OHLCV data from the frontend to the FastAPI backend, we were able to prompt the LLM to generate real-time, context-aware technical analysis and display it in a dynamic side-panel for the user.

Data Sourcing Strategy: To maintain a cost-effective development cycle and create a viable MVP prototype without the heavy initial overhead of premium institutional data feeds, we relied on alternative data aggregation methods. By utilizing the Python yfinance library, we effectively scraped and proxied Yahoo Finance data. This approach allowed us to

fetch historical date ranges, search for US equities and ETFs, and provide realistic market data to both our interactive charts and our backtesting engine entirely for free.

To provide a realistic, risk-free execution engine for our Paper Trading tab, we integrated the Alpaca Trading API via a secure Python abstraction layer (alpaca_config.py) on our FastAPI backend. This architecture ensures that sensitive API credentials remain hidden from the frontend while exposing streamlined endpoints for the React dashboard to seamlessly manage account equity, monitor active positions, and execute market orders. Furthermore, we leveraged the Alpaca Data API to fetch high-resolution trade ticks and 1-minute OHLCV bars, allowing our backend algorithms to calculate and serve advanced, institutional-grade Volume Profile indicators directly to our interactive charts. Ultimately, this creates a cohesive workflow where users can perform granular technical analysis and instantly execute paper trades reflecting live market conditions without ever leaving the application.

5 Vibe Coding Process

This was the most educational part of the project. I will break it into four sub-topics: workflow, what worked, what struggled, and takeaways.

5.1 Workflow

The platform was built almost entirely through Google's Antigravity, and Claude Code, Anthropic's agentic coding tool that runs in the terminal and edits files directly. My workflow was:

1. Write a feature request in natural language in a chat window (e.g., "Add a highlight settings modal where users can define keyword categories with colors"). Besides natural language for prompting, we occasionally used code snippets for examples and templates for the models to generate.

2. Let Claude Code and Codex implement it across `App.js`, `App.css`, and any new utility files.

3. Review the diff, run the app locally, test the feature.

4. Iterate if something broke or looked wrong, usually with targeted follow-ups like "The modal is cut off on mobile, cap height at 80% viewport and add overflow scrolling."

We also used two utilities for the agents to keep track of the context of the project. One was an AGENTS.md file, quite similar to a CLAUDE.md file; having this file allowed us to provide persistent context to the models as well as maintain a compatibility layer between Anthropic's models and other agentic-driven development ecosystems. The following global rules context was also utilized to help the models acclimate to our context and development:

- My OS is **Windows**.

- Use `CMD` or `Powershell` command syntax.
- Please update the `agents.md` file (if there exists one) after substantial changes.
- Act as a **senior software engineer** who avoids code smells and prioritizes writing extensible and modular code.

5.2 Strengths

We found that the models we used were strong at a multitude of tasks: (1) **feature scaffolding** – adding a new tab, modal, or button with handlers wired up correctly; (2) **React patterns** – hooks, state lifting, context usage, and memoization were near-perfect; (3) **CSS and responsive layout** – Claude was strong at writing flexbox and grid layouts that worked across breakpoints, and at matching existing design tokens; (4) **boilerplate-heavy tasks** – Firestore queries, API proxy endpoints, CSV exports, all of which would have been tedious to write by hand.

We found that different models have differing proficiencies at tool-calling, which is a huge part of agents and vibe coding. We found that, for many models, tool-calling is generally more accurate compared to six months ago. We also found that models are now better at testing features themselves. For example, Claude and Gemini are both capable of launching automated browser sessions to test features. However, therein is also a weakness, since this caused some inconsistencies with the web app sometimes having crashes because of the model's CLI permissions.

5.3 Weaknesses

The models sometimes failed to recognize the nature of our backend as a Python server. When the model updated the backend, the app would crash due to the backend Python script not having hot-reloading compared with the Vite React frontend part of the app. This was a minor inconvenience but still a notable drawback of our workflow. Moreover, this is part of a bigger issue where models and AI generally struggle with complex state synchronization.

The pattern across all three: Claude Code excels when given a concrete, scoped task with clear inputs and outputs. It struggles when the problem requires holistic judgment or runtime observation it cannot perform on its own.

5.4 Takeaways

We found agentic driven vibe coding to be worth the time and faster in speed compared to regular coding. The models were generally proficient at coding for all tasks we assigned them.

Three things I learned about vibe coding:

- `CLAUDE.md` **is non-negotiable.** A well-maintained project-context file cuts prompt length in half and improves first-pass correctness dramatically.

- **Small, scoped prompts beat big vague ones.** "Add keyword highlighting" produced mediocre first drafts; "Add keyword highlighting as `<mark>` tags inside the abstract component, with colors from the `keywordCategories` state, toggleable via a button" produced near-final code.

- **Stay in the loop on architecture.** AI is a faster typist than a better architect. I made the structural decisions (single-file `App.js`, dual-layer persistence, serverless proxies) and delegated implementation – not the other way around.

6 Roadmap and Conclusion

For future development and features, we want to transition away from our current free model of data-acquisition to a more professional system. We want to get institutional level data that generally improves user experience and brings it in line with what people would experience actually trading on the stock market with real money.

We would also want to move towards users being able to integrate their actual brokerage accounts and trade on our app that way in the future. In our current concept we decouple the learning process from the financial risk of active trading and in doing so we create a safe sandbox for investor growth.

The future evolution of the project is twofold. On the educational side, we plan to expand the asset library and deepen the granularity of our AI explanations. On the technical side, our predictive models and risk analysis systems would be iterated upon. Ultimately, we aim to scale from a curated list of stocks to a universal market-viewing tool that empowers users to transition from observers to informed, confident strategists.

Moreover, we want to focus on becoming a dual-purpose application for both learning and testing/deploying strategies. By more tightly integrating our ideas for

Live Demo: `https://genai-trading.vercel.app/`
Repository: `https://github.com/jl0905/genai-trading`